ЛАБОРАТОРНАЯ РАБОТА №3. РАЗРАБОТКА АГЕНТОВ ОБРАБОТКИ ЗНАНИЙ СРЕДСТВАМИ SPADE И ТЕХНОЛОГИИ OSTIS

Цель

Изучить принципы агентно-ориентированного подхода и асинхронной обработки знаний в интеллектуальных системах. Приобрести практические навыки разработки взаимодействующих агентов на платформе SPADE с использованием RDF и SPARQL, программирования на Языке SCP, создания sc-агентов и формализации их спецификаций и компонентов пользовательского интерфейса для их вызова.

Теоретические сведения

Разделы учебного пособия:

- ЛР 1 (Глава 1, Глава 3, Глава 5 (до Параграфа 5.7), 6 (до Пункта 6.6.2.9), Глава 7 (до Параграфа 7.6, Параграф 7.7, Разделы 7.9.1.2.3 – 7.9.1.2.4))
- + ЛР 2 (Глава 2, Глава 6 (Пункт 6.6.2.10), Глава 7 (Разделы 7.6.1 – 7.6.6, Разделы 7.9.2.3-7.9.3))
- + Глава 5 (Параграф 5.7), Глава 7 (Разделы 7.6 – 7.9.1.3.2.4), Глава 8
- + Глава 7 (Раздел 7.9.2)*.

Индивидуальное задание

1. Продолжить работу с предметной областью и базами знаний, разработанными в лабораторной работе №1 и дополненными в лабораторной работе №2. Использовать RDF/OWL-представление для части задания по SPADE и sc-представление для части задания по OSTIS. При необходимости дополнить обе модели согласованными тестовыми данными.
2. Подготовить общую постановку задачи:
    a. выбрать информационную потребность из лабораторной работы №2 либо сформулировать на её основе составную задачу поиска и обработки знаний;

* — материал для углубленного изучения темы.

b. определить входные параметры, требования к ним, дополнительное условие отбора и ожидаемый результат; предусмотреть случаи непустого и пустого результатов, а также некорректного аргумента;
c. выбрать не менее двух SPARQL-запросов и не менее двух sc-шаблонов из лабораторной работы №2, используемых при решении задачи; при необходимости адаптировать их и описать изменения;
d. обеспечить эквивалентность постановки задачи в обеих технологиях; составить таблицу соответствия используемых классов, отношений, параметров и идентификаторов;
e. указать, используются ли только явно заданные факты или также результаты логического вывода; если необходимы выведенные факты, описать их получение и обеспечить сопоставимость исходных данных.

3. Используя средства Технологии Semantic Web и платформу SPADE:
   a. изучить архитектуру SPADE, жизненный цикл агентов, типы поведений и принципы асинхронного обмена сообщениями;
   b. подготовить Python-окружение, установить SPADE и RDFLib, настроить XMPP-сервер и учётные записи агентов; зафиксировать версии компонентов, зависимости и порядок запуска; пароли хранить вне исходного кода;
   c. загрузить RDF-данные из лабораторной работы №1 с дополнениями лабораторной работы №2 средствами RDFLib; проверить доступность выбранных SPARQL-запросов и сохранить контрольные результаты их отдельного выполнения;
   d. разработать не менее двух взаимодействующих агентов:
      i. агент-клиент, принимающий параметры задачи, отправляющий запрос и представляющий пользователю полученный результат;
      ii. агент обработки знаний, принимающий запрос, проверяющий его параметры, выполняющий поиск и дополнительную обработку RDF-данных и отправляющий ответ.
   e. для каждого агента определить JID, назначение, входные и выходные сообщения, используемые поведения и условия завершения; представить схему взаимодействия агентов;
   f. реализовать агентную систему на Python с использованием SPADE:

* — материал для углубленного изучения темы.

i. предусмотреть однократную отправку запроса с использованием OneShotBehaviour и циклическую обработку входящих запросов с использованием CyclicBehaviour;
ii. реализовать в агенте обработки знаний не менее двух параметризованных SPARQL-запросов на основе результатов лабораторной работы №2; передавать значения параметров через связывание переменных либо эквивалентный безопасный механизм;
iii. объединить результаты запросов в общий алгоритм с проверкой дополнительного условия и формированием итогового ответа;
iv. использовать Message для передачи сообщений и Template для их маршрутизации; задать согласованные метаданные performative, ontology и protocol;
v. задать формат тела сообщения и правила передачи параметров и результатов; для структурированных параметров допускается JSON, для передачи RDF-фрагмента — Turtle с указанием типа содержимого;
vi. связывать ответ с исходным запросом при помощи thread или другого явно описанного идентификатора; проверять отправителя ответа;
vii. установить ограниченное время ожидания ответа; различать успешный непустой ответ, успешный пустой ответ, отказ из-за некорректных параметров, ошибку обработки и истечение времени ожидания;
viii. обеспечить корректный запуск и остановку агентов и ведение журнала запросов, ответов и ошибок.

g. описать протокол взаимодействия: последовательность сообщений, обязательные поля, статусы и действия агентов при ошибках; метаданные и формат тела сообщения должны проверяться в соответствии с этим протоколом;
h. реализовать способ пользовательского вызова: консольную команду или простую форму, передающую параметры агенту-клиенту и отображающую ответ; встроенный веб-интерфейс мониторинга SPADE не считать заменой интерфейсу постановки задачи;

* — материал для углубленного изучения темы.

i. выполнить разработанную систему, сохранить исходные тексты, примеры сообщений, журнал выполнения и полученные результаты.

4. Используя средства Технологии OSTIS, разработать SCP-программу:
   a. изучить принципы решения задач в ostis-системах, структуру SCP-программ, назначение операторов, операндов и связей управления;
   b. использовать пример ostis-системы из лабораторной работы №2; зафиксировать его версию или ревизию и используемую реализацию интерпретатора SCP;
   c. для общей задачи разработать алгоритм и определить интерфейс программы: системный идентификатор, входные и выходные параметры, их типы и условия использования;
   d. представить SCP-программу на SCs-коде; задать параметры, начальный оператор, операнды и переходы;
   e. реализовать операции поиска на основе не менее двух sc-шаблонов из лабораторной работы №2; установить соответствие элементов шаблонов операндам операторов;
   f. предусмотреть условное ветвление, перебор кандидатов до исчерпания результатов, проверку дополнительного условия и формирование результирующей sc-конструкции;
   g. обеспечить завершение программы при непустом и пустом результатах, сохранность исходных знаний и освобождение временных sc-конструкций;
   h. подготовить отдельный сценарий запуска программы, выполнить его в примере ostis-системы и сохранить результат.
5. Используя агентно-ориентированный подход в ostis-системах:
   a. изучить принципы разработки sc-агентов и определить класс действий для решения общей задачи, аргументы действия и ожидаемый результат;
   b. формализовать на SCs-коде спецификацию sc-агента: первичное условие инициирования, условие инициирования и ожидаемый результат, класс действий и ключевые sc-элементы;
   c. разработать программу реализации sc-агента на Языке SCP и представить её на SCs-коде; обеспечить получение действия и аргументов, их проверку, вызов ранее разработанной SCP-программы и ожидание завершения вызова;

* — материал для углубленного изучения темы.

d. переиспользовать основную SCP-программу без дублирования её алгоритма; связать результат с действием и зафиксировать успешное либо неуспешное завершение;
e. предусмотреть контролируемую обработку отсутствующего обязательного аргумента и аргумента недопустимого класса; отличать ошибку от корректного выполнения с пустым результатом;
f. связать спецификацию с реализацией и подключить агента к примеру ostis-системы;
g. описать на SCs-коде команду пользовательского интерфейса: идентификатор, название, шаблон действия и соответствие аргументов команды аргументам действия;
h. подключить команду к меню, описать порядок её вызова и проверить получение результата пользователем.

6. Протестировать обе реализации:
   a. подготовить не менее шести общих тестовых сценариев:
      i. корректные аргументы и несколько подходящих объектов;
      ii. корректные аргументы и отсутствие подходящих объектов;
      iii. наличие кандидатов, часть которых исключается дополнительным условием;
      iv. отсутствие обязательного аргумента;
      v. недопустимый тип или класс аргумента;
      vi. повторный запрос с теми же аргументами без изменения исходных данных.
   b. для каждого сценария зафиксировать исходные данные, ожидаемый результат или статус, фактические результаты в SPADE и OSTIS и вывод о корректности;
   c. дополнительно для SPADE проверить отсутствие ответа в установленное время, а также не менее двух запросов с различными thread; убедиться в правильном сопоставлении ответов запросам;
   d. сопоставить ответы SPADE с отдельным выполнением используемых SPARQL-запросов, а результат sc-агента — с отдельным запуском SCP-программы и поиском по sc-шаблонам;
   e. не менее двух общих сценариев выполнить через пользовательский способ вызова каждой реализации; сохранить сообщения, журналы, скриншоты либо sc-конструкции, подтверждающие результат;
   f. проверить сохранность исходных знаний, отсутствие зависания и корректное завершение компонентов.

* — материал для углубленного изучения темы.

7. Сопоставить агентные реализации средствами SPADE и OSTIS:
    a. сравнить представление знаний, способы инициирования обработки, передачу параметров, управление вычислением, асинхронное взаимодействие и представление результата;
    b. установить соответствие между сообщением-запросом SPADE и инициируемым действием OSTIS, ответом агента и результатом действия; указать границы такого сопоставления;
    c. сравнить результаты не менее трёх общих сценариев по содержанию и полноте; при различиях пояснить влияние исходных фактов, логического вывода и условий поиска;
    d. указать, какие элементы из лабораторных работ №1 и №2 были переиспользованы и какие дополнения внесены;
    e. сформулировать вывод о переходе от модели предметной области и поисковых операций к агентной обработке знаний; непосредственный обмен сообщениями между SPADE и OSTIS в данной работе не требуется;
    f. представить исходные файлы обеих реализаций, базы знаний, спецификации, пользовательские команды, тесты, зависимости и инструкции запуска.

8*. Разработать sc-агент на языке C++ для той же задачи, используя Технологию OSTIS:
    g. изучить C++ API sc-machine для работы со SC-памятью, действиями, агентами, ключевыми sc-элементами и модулями; использовать API, соответствующий зафиксированной версии системы;
    h. подготовить модуль решателя задач: исходные и заголовочные файлы, описание зависимостей и CMakeLists.txt; зафиксировать компилятор, стандарт C++, версии sc-machine, CMake и используемого менеджера зависимостей;
    i. определить ключевые sc-элементы: класс действий, классы допустимых аргументов и используемые отношения; согласовать их системные идентификаторы с базой знаний и описать средствами ScKeynodes/ScKeynode либо соответствующего API выбранной версии;
    j. реализовать класс агента, обрабатывающего инициированное действие; для API учебного пособия использовать

* — материал для углубленного изучения темы.

ScActionInitiatedAgent и определить GetActionClass и DoProgram; при отличиях версии указать эквивалентные классы и методы;

k. получать аргументы через API действия, проверять существование sc-элементов, их типы и принадлежность требуемым классам; некорректные аргументы обрабатывать до основного поиска;
l. реализовать алгоритм непосредственно на C++ средствами контекста SC-памяти, поиска по sc-шаблонам и/или итераторов; воспроизвести не менее двух поисковых операций лабораторной работы №2, дополнительную проверку и формирование результата; простой вызов готовой SCP-программы не считать выполнением реализации на C++;
m. сформировать результирующую sc-структуру, включить необходимые узлы и соединители, связать результат с действием через API и установить конечный статус; отдельно обработать корректный пустой результат и ошибку;
n. зарегистрировать агента в модуле, собрать модуль и подключить его к примеру ostis-системы; сохранить команды сборки и запуска и подтверждение успешной загрузки;
o. формализовать спецификацию C++-агента на SCg-коде или SCs-коде и указать её связь с программной реализацией; согласовать аргументы и смысл результата со спецификацией SCP-агента;
p. исключить одновременную обработку одного действия двумя эквивалентными реализациями: использовать разные классы действий с одинаковой семантикой либо попеременно активировать SCP-агент и C++-агент; явно описать выбранную схему;
q. описать отдельную пользовательскую команду C++-агента либо порядок переключения реализации существующей команды; проверить вызов через интерфейс и соответствие аргументов;
r. разработать автоматические тесты средствами Google Test и тестового контекста SC-памяти либо эквивалентных средств выбранной версии; в тестах инициировать действие и проверять его конечный статус и содержимое результата, а не только отсутствие исключений;
s. покрыть автоматическими тестами не менее шести общих сценариев следующего пункта; обеспечить независимость тестов и

* — материал для углубленного изучения темы.

отсутствие влияния данных предыдущего запуска; сохранить исходные тесты и отчёт об их выполнении.

Контрольные вопросы:

Вопросы вида “Что такое”, “В чём отличие между”.

* — материал для углубленного изучения темы.